

TQEx: Tensor-based Query Engine Enhanced by Bridging the Gap

HAITAO ZHANG, School of Computer Science, Wuhan University, China

RAN PANG, School of Computer Science, Wuhan University, China

YUANYUAN ZHU*, School of Computer Science, Wuhan University, China

HAO ZHANG*, Huawei Technologies, China

CONGLI GAO, Huawei Technologies, China

MING ZHONG, School of Computer Science, Wuhan University, China

JIawei JIANG, School of Computer Science, Wuhan University, China

TIEYUN QIAN, School of Computer Science, Wuhan University, China

JEFFREY XU YU, The Hong Kong University of Science and Technology (Guangzhou), China

With the development of AI and the growing demand for computational power, hardware is becoming increasingly specialized and heterogeneous. The emergence of diverse specialized hardware architectures, each with distinct characteristics and programming abstractions, poses significant portability and sustainability challenges for existing data processing systems. Tensor Computation Runtimes (TCRs) abstract away the low-level hardware complexities by providing users with a hardware-independent tensor-based interface, enabling data scientists to effectively leverage the powerful capabilities of new hardware accelerators (collectively referred to as XPU). Built on TCRs, the existing relational query engine TQP demonstrates portability across a wide range of target hardware and sustainability along with the ongoing evolution of TCRs and hardware. However, it neglects the big gap between irregular SQL workloads and uniform tensor operations when mapping SQL operators to tensor programs, which causes significant storage and computation overhead. In this paper, for the first time, we analyze the underlying gap between SQL and tensors, and provide guidelines to bridge it. Following these guidelines, we build a new Tensor-based Query Engine Enhanced (TQEx) by bridging the gap from multiple aspects: develop efficient storage and computation strategies for variable-length data, and design efficient SQL operators such as join and aggregate based on tensors. We also extend TQEx to multi-XPUs for large-scale data processing. Extensive experimental studies show that our query engine, TQEx, achieves a 9.6 $\times$ speedup (with a peak of 41.9 $\times$) over TQP on TPC-H, and it is also 27.9 $\times$ faster than leading GPU databases such as HeavyDB. On TPC-H at scale factor 100, TQEx outperforms DuckDB by 12.2 $\times$ and HeavyDB by 22.7 $\times$ on supported queries.

CCS Concepts: • **Information systems** → **Data management systems**.

Additional Key Words and Phrases: Relational Query Engine, SQL Operators, Tensor Operations

*Yuan Yuan Zhu and Hao Zhang are corresponding authors.

Authors' Contact Information: Haitao Zhang, School of Computer Science, Wuhan University, China, htzhang@whu.edu.cn; Ran Pang, School of Computer Science, Wuhan University, China, pangran@whu.edu.cn; Yuan Yuan Zhu, School of Computer Science, Wuhan University, China, yyzhu@whu.edu.cn; Hao Zhang, Huawei Technologies, China, zhanghao687@huawei.com; Congli Gao, Huawei Technologies, China, gaocongli@huawei.com; Ming Zhong, School of Computer Science, Wuhan University, China, clock@whu.edu.cn; Jiawei Jiang, School of Computer Science, Wuhan University, China, jiawei.jiang@whu.edu.cn; Tieyun Qian, School of Computer Science, Wuhan University, China, qty@whu.edu.cn; Jeffrey Xu Yu, The Hong Kong University of Science and Technology (Guangzhou), China, jeffreyyxuyu@hkust-gz.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2025/12-ART370

<https://doi.org/10.1145/3769835>

ACM Reference Format:

Haitao Zhang, Ran Pang, Yuanyuan Zhu, Hao Zhang, Congli Gao, Ming Zhong, Jiawei Jiang, Tiejun Qian, and Jeffrey Xu Yu. 2025. TQEx: Tensor-based Query Engine Enhanced by Bridging the Gap. *Proc. ACM Manag. Data* 3, 6 (SIGMOD), Article 370 (December 2025), 27 pages. <https://doi.org/10.1145/3769835>

1 Introduction

Nowadays, data volume is growing at an unprecedented rate in both businesses and scientific communities [25, 69, 70], while the performance improvement of a single CPU has slowed down due to physical limitations [74]. To keep up with the growth of data, hardware manufacturers have shifted their focus to multi-core CPUs and specialized hardware accelerators with a large number of cores such as Graphics Processing Units (GPUs), Tensor Processing Units (TPUs), Neural Processing Units (NPU), and Field-Programmable Gate Arrays (FPGAs). Thus, workloads can be distributed across multiple cores for parallelism, thereby increasing the overall performance [11].

Data processing systems have been embracing emerging hardware for acceleration. Earlier works mainly leverage multi-core CPUs to develop accelerated databases [40, 59] or optimize individual SQL operators (join [7, 9, 36, 62, 73], sort [18, 33, 55], and aggregate [20, 81]). Over the past decade, GPUs have been used as co-processors for query acceleration [13, 19, 27, 30, 41, 43, 52, 61, 76, 82, 83]. With the increasing memory capacity of GPUs, some recent works directly store and compute the whole or a significant fraction of the dataset on the GPU, including commercial query engines such as HeavyDB [2], BlazingSQL [1], and Kinetica [3], as well as open-source query engines such as GaccO [10], Crystal [64] and its extension [65]. Cao et al. [14] analyze the cross-stack performance and resource utilization of these GPU systems on the Star Schema Benchmark (SSB). TCUDB [31] exploits the power of Tensor Core Units (TCUs) by transforming queries into matrix operations. Other works have also explored FPGA-accelerated databases in many aspects [50], including complex query computation [84], distributed computation [34], and benchmarking [77].

However, the above engines are tailored for specific types of hardware accelerators (e.g., existing GPU systems for CUDA-based NVIDIA GPUs but not for AMD GPUs and Apple chips), which leads to endless redevelopment to keep up with emerging hardware. Although *composable databases* that can be assembled from reusable high-quality components have been proposed to prevent redeveloping tightly integrated systems in new domains [8, 35, 39, 53], they mainly focus on providing modular components and open standards. These approaches, however, do not address how to achieve system reusability across heterogeneous specialized hardware.

To achieve cross-hardware reusability, one possible way is to build a high-level framework that abstracts away the underlying specialty of heterogeneous hardware backends [66]. TQP [29] is such a query engine built on Tensor Computation Runtimes (TCRs), which refer to the DL frameworks (e.g., PyTorch [51], TensorFlow [6], and MXNet [16]) along with their compilers and runtimes (e.g., TVM [17], and ONNX runtime [45]). TCRs provide tensor-based, hardware-independent programming interfaces on top, and a runtime and a dispatcher/compiler at the bottom to enable running programs over heterogeneous hardware accelerators (e.g., GPU, TPU, NPU, collectively referred to as XPU). TQP inherits the portability from TCRs, and has developed algorithms to map each SQL operator to a series of tensor operations. Nevertheless, TQP [29] involves significant overhead when mapping SQL operators to tensor programs, as it neglects the following significant gap between SQL and tensors inherited from their disparate characteristics.

Irregularity in SQL vs. Uniformity in Tensors. Relational tables usually contain variable-length data (e.g., strings), and each tuple may require varying types and amounts of computation in some SQL operators (e.g., JOIN, CASE, and LIKE) [28, 67, 75], exhibiting irregularities in both data and computation, while tensors prefer uniform storage and computation. Thus, TQP [29] adopts padding to handle such irregularities. For data irregularity, it uses storage padding to append

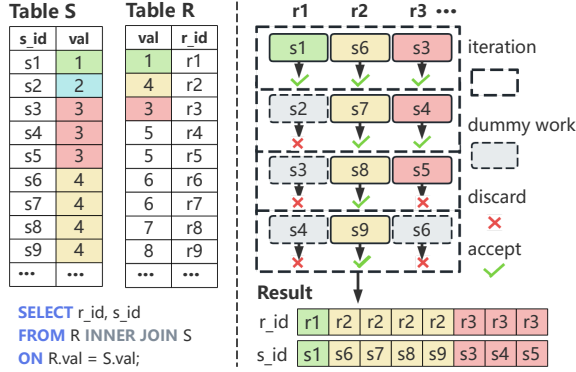


Fig. 1. Loops with irregularity in join

dummy elements, such as zero, to reach a uniform maximum length. For computation irregularity, it uses computation padding to transform the diverging computation flow of tuples into uniform ones, and then discards the padding results later for correctness. As shown in Fig. 1, in the JOIN operation, tuples s_1 , s_2 , and s_3 have 1, 4, and 3 matched tuples, respectively. The computation flow of each tuple is extended to 4 iterations by padding computation indicated by the gray blocks. When the types of operations differ across tuples, such as CASE and LIKE, all tuples are extended to execute all possible types of operations, i.e., if there are t operation types, each tuple may be padded with the remaining $t - 1$ operations as dummy ones. However, the extensive use of padding introduces significant storage and computation overhead, ultimately degrading system performance.

Loops in SQL vs. Loop-free Preferred by Tensors. Many SQL operators (e.g., JOIN and LIKE) involve loops, where each tuple requires iterative computation. As illustrated in Fig. 1, after computation padding, such loops are transformed into four global rounds, where each round (indicated by a dashed block) processes all tuples simultaneously by a series of tensor operations. However, loops in tensor programs will cause frequent kernel launches associated with each tensor operation [27, 57], and frequent materialization of large intermediate results from cache to global memory [38, 42, 58, 60]. Moreover, loops will also disrupt spatial locality. As shown in Fig. 1, tuple r_3 requires s_3 , s_4 , and s_5 in rounds 1, 2, and 3, respectively, which are stored consecutively in S, but their sequential access is disrupted by loops. Due to significant overhead caused by loops, efficient tensor programs usually prefer a loop-free structure. Note that although loops can be further optimized during compilation [17, 42, 51], this optimization is not applicable for loops in online SQL queries where the iteration number remains unknown until execution.

The above two issues are usually compounded in complex SQL operators such as JOIN, as illustrated in Fig. 1, which becomes a severe bottleneck of tensor-based relational query engines.

In this paper, we formally analyze the gap between SQL and tensors, and propose two guidelines to bridge it. (1) *Reduce/Eliminate Irregularity*. Reduce the effect of irregularity by early stopping the padded computation, or eliminate the irregularity by reformulating the computation into regular forms when possible. (2) *Unroll Loops*. Transform loops into large batch tensor operations to reduce kernel launches and materialization while preserving spatial locality. Based on these guidelines, we design a new Tensor-based Query Engine Enhanced (TQEx) by bridging the gap between SQL and tensors, which can support the foundational SPJ (Select-Project-Join) queries, as well as commonly used aggregate and string operators. Specifically, we focus on optimizing two key parts: the storage and manipulation of variable-length strings, and the computation of complex SQL operators with irregularity and loops such as JOIN and AGGREGATE. We also extend our method to multi-XPUs since the memory capacity of an XPU is usually too limited to support large-scale data processing.

We conducted extensive experimental studies to evaluate the performance of TQEx by comparing it with the state-of-the-art CPU system (DuckDB) and GPU systems (HeavyDB, BlazingSQL, and TQP) on TPC-H with different scale factors (SF). Our query engine TQEx achieves a speedup of $9.6\times$ over TQP, and is $27.9\times$ faster than the leading GPU database HeavyDB. Moreover, TQEx on multi-XPUs achieves speedups of $12.2\times$ and $22.7\times$ over DuckDB and HeavyDB, respectively, and TQEx can successfully answer all the TPC-H queries at SF 300 while HeavyDB fails on a subset of them. We also validated the portability of TQEx by deploying it on different types of hardware accelerators, including NVIDIA V100, AMD 7700XT, and Apple M2 Pro chip, where AMD 7700XT achieves the best performance on most queries. Our contributions are summarized as follows.

- We formally analyze the underlying gap between SQL and tensors, and provide guidelines to bridge this gap for the first time. Our analysis and guidelines can benefit other tensor-based data processing tasks with similar issues.
- Following the guidelines, we build a new query engine TQEx enhanced by bridging the gap between SQL and tensors from multiple aspects, along with extensions to multi-XPUs.
- We conducted extensive experimental studies, which validated: the outperformance of TQEx over state-of-the-art CPU and GPU systems, the scalability on different scale factors and number of GPUs/cores, the portability across heterogeneous hardware backends, and the detailed performance gains by performance breakdown and ablation study.

Roadmap. Section 2 presents the preliminaries. Section 3 analyzes the gap between SQL and tensors and provides guidelines to bridge it. Section 4 illustrates how to efficiently store and operate strings according to the guidelines. Section 5 discusses how to efficiently implement join and aggregate based on the guidelines and extend our method to multi-XPUs. Section 6 shows the experiments, and Section 7 reviews the related works. Section 8 concludes the paper.

2 Preliminary

In this section, we introduce basic tensor operations and review the state-of-the-art tensor-based query engine TQP [29]. Table 1 summarizes the key symbols used in this paper.

2.1 Tensor Operations

A tensor is a multi-dimensional array, where a scalar is a 0-dimensional tensor, a vector is a 1-dimensional tensor, and a matrix is a 2-dimensional tensor. TCRs provide a set of tensor operations for users to write general tensor programs that can be efficiently dispatched to a variety of specialized hardware backends. The common tensor operations can be categorized as follows. (1) *Initialization*: create a tensor with specific values from another tensor or other data formats such as NumPy and lists, e.g., ones, zeros, tensor. (2) *Selection*: choose elements from a tensor based on index/mask or slicing, e.g., index_select, masked_select. (3) *Comparison*: compare a tensor with another tensor, e.g., eq, lt, gt, le, ge. (4) *Arithmetic*: perform basic arithmetic operations on tensors, e.g., add, mul, div, sub. (5) *Reorganization*: rearrange elements in a tensor or concatenate multiple tensors, e.g., sort, cat, stack. (6) *Reduction*: aggregate over a tensor or get unique values from a tensor, e.g., sum, max, min, mean, scatter_max, scatter_min, scatter_add, unique.

TCRs also provide the broadcast mechanism for some tensor operations to automatically expand the tensor arguments to equal sizes without making copies of the data [5]. For an operation between an $n \times m$ matrix and a 1-dimensional row tensor with length m (or a column tensor with length n), the 1-dimensional tensor is implicitly replicated along the column (or row) dimension without physical copying. Similarly, for an operation between a row tensor of size m and a column tensor of size n , both tensors will be broadcast along the column or row, respectively. This broadcast mechanism can reuse data and preserve computational locality during tensor computation.

Table 1. Summary of symbols

Symbol	Description
str_set	the concatenated strings in a column
$start, len$	start indices and lengths of all the strings in str_set , respectively
$pattern$	a string pattern of length k
S, R	build table and probe table
ID_S, ID_R, key_S, key_R	tuple IDs and join keys of table S and R , respectively
$C_S/M_S, C_R/M_S$	candidate/matched tuple IDs of S and R in join, respectively
$cdata$	data of the aggregate column
$groupID, count$	group ID of each tuple and the number of tuples in each group

2.2 Existing Tensor-based Query Engine TQP

TQP [29] adopts storage padding for variable-length strings and computation padding for irregular workloads in SQL operators. Note that padding for irregularity has also been discussed in vectorized engines [44, 47] with the focus on whether and how to use padding to implement vectorized operations, which differs from the tensor-based system that focuses on reducing the effect of padding required by the uniformity of tensors.

Storage Padding for Strings. TQP stores a column of m strings with variable length as a 2-dimensional tensor M_{str} , i.e., an $m \times l_s$ matrix where l_s is the maximum string length. Each row in M_{str} represents a string, and each element represents a character. A string with length smaller than l_s is right-padded with 0s. While such padding simplifies tensor-based string manipulations, it also causes significant overhead of storage space (almost doubles the storage of strings in the TPC-H dataset). Moreover, excessively padded elements will be processed and then discarded during the computation, leading to substantial computation overhead.

Computation Padding for Join. TQP implements hash join based on tensors, which is similar to the classical hash-join algorithm, except that the *build* and *probe* phases are interleaved and repeated b_{max} times, where b_{max} is the maximum number of elements in a hash bucket. In each iteration, TQP first *builds* a 1-dimensional hash table $htable$ of length $h + 1$ (including an extra dummy slot) by mapping tuples of the smaller (build) table S to this vector, where tuples with the same hash value i randomly overwrite each other at $htable[i]$. Then, each tuple in the larger (probe) table R probes $htable$ to find the matched tuples in S . Note that tuples in S that have been used will be preserved and remapped to the dummy slot with hash value h in subsequent iterations to avoid duplication. However, remapping each tuple in S to the hash table and matching each tuple in R against the hash table for b_{max} times results in excessive computation padding, which severely degrades performance, especially when the distribution of hash values is highly skewed.

Overall, the storage and computation padding strategies in TQP cause significant space and computation overhead. The underlying reason is that TQP neglects the large gap between SQL and tensors stemming from their disparate characteristics. In this paper, for the first time, we carefully analyze the characteristics of SQL and tensors, and provide solutions to bridge this gap.

3 Analyzing and Bridging the Gap

We first show the gap between SQL and tensors through statistical analysis, and then provide guidelines to bridge it.

3.1 Analysis of the Gap Between SQL and Tensors

Characteristics of SQL. Relational databases usually exhibit irregularities in both data and computation. We statistically show such irregularity in the TPC-H benchmark. (1) *Data irregularity.* Take column O_COMMENT of table ORDERS in TPC-H as an example. Fig. 2a shows that 98.3%

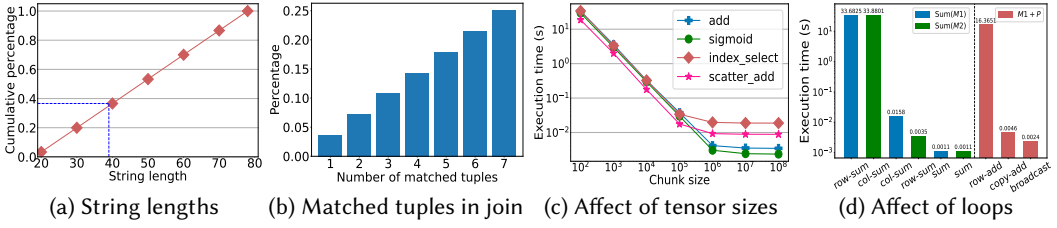


Fig. 2. Statistical analysis of SQL and tensors

strings have lengths smaller than the maximum string length, with 38.2% strings below half of the maximum, resulting in significant storage overhead if they are padded to the maximum length. (2) *Computation irregularity*. Take the self-join of table LINEITEM on column L_ORDERKEY in TPC-H as an example. Fig. 2b shows significant variations in the number of matched tuples. Data irregularity and computation irregularity may be nested together in complex operators such as LIKE in string matching, where each string involves different numbers of iterations and performs divergent execution in each iteration.

Characteristics of Tensors. As the minimal programming unit in tensor programs, a tensor operation performs uniform computation across all elements of the entire input tensors. To fully leverage the parallelism of tensor operations, tensor programs should have the following characteristics.

(1) *Prefer large tensors*. Due to the substantial kernel launch overhead associated with tensor operations [27, 57], efficient tensor programs prefer to process a large tensor in a batch rather than process fragmented small tensors multiple times. We illustrate this with four tensor basic operators. We divide a 1-dimensional tensor of length 10^8 into chunks with sizes ranging from 10^2 to 10^7 and sequentially process each chunk. Fig. 2c shows that the execution time decreases as the chunk size increases and achieves the minimum when processing the tensor as a whole. For tensors larger than 10^6 , kernel launch overhead becomes negligible compared to the computational workload.

(2) *Prefer loop-free structure*. Loops in tensor programs usually cause significant overhead, including frequent kernel launches, materialization of large intermediate results, and disruption of computation locality. We take the sum over a matrix and the addition of a vector to a matrix as examples to show the effect of loops in Fig. 2d. On the left, we compare three implementations for summing all elements in a $10^6 \times 100$ matrix M_1 and a 100×10^6 matrix M_2 : row-sum to iteratively compute the sum of each row first and then aggregate them; col-sum to iteratively compute the sum of each column first and then aggregate them; sum to compute the sum of all the elements in a matrix simultaneously. row-sum on M_1 and col-sum on M_2 both require 10^6 iterations and take approximately 33s due to frequent kernel launches. col-sum on M_1 and row-sum on M_2 both require 100 iterations with negligible kernel launch but still take 17.6ms and 3.7ms, respectively, which are longer than 1.2ms achieved by sum, due to the costly materialization of large intermediate results. Moreover, col-sum is 4.75 $\times$ slower than row-sum as extracting columns from the default row-based storage disrupts locality. The single sum operation on M_1/M_2 achieves the best performance by eliminating frequent kernel launches and excessive intermediate result materialization while preserving locality. On the right, we compare three implementations for adding a vector P of length 100 to each row of M_1 : row-add that repeatedly scan P and add it with each row of M_1 , completing in 15.75s; copy-add that stacks 10^6 copies of P and then performs a single element-wise addition operation with M_1 , completing in 2.3ms; broadcast-add that leverages broadcast to logically replicate P to each row of M_1 without physical copying, completing in 1.2ms.

The Gap between SQL and Tensors. Based on the above analysis, the gap between SQL and tensors is clear: (1) *The data and computation in SQL queries are usually irregular, while tensors prefer*

Algorithm 1: candidate ID pairs generation in hash join

input : ID_S of table S , ID_R of table R , start positions and lengths of hash buckets, $start_R$ and len_R respectively

output: candidate ID pairs C_S and C_R

- 1 $l_{\max} \leftarrow \max(len_R)$; $increment \leftarrow \text{arange}(l_{\max})$;
- 2 $idx_R \leftarrow start_R.view(-1, 1) + increment$;
- 3 $mask \leftarrow increment < len_R.view(-1, 1)$;
- 4 $idx_R \leftarrow \text{masked_select}(idx_R.view(-1), mask.view(-1))$;
- 5 $C_S \leftarrow ID_S[idx_R]$;
- 6 $ID_R \leftarrow \text{arange}(|R|)$;
- 7 $C_R \leftarrow \text{repeat_interleave}(ID_R, len_R)$;
- 8 **return** C_S, C_R ;

uniform storage and computation. (2) SQL operators usually involve loops, while tensor programs prefer loop-free control flows. This gap poses significant challenges in converting irregular SQL queries to loop-free tensor programs. We provide guidelines on how to bridge this gap in the following.

3.2 Guidelines for Bridging the Gap

Now, we provide two general guidelines for bridging the gap between SQL and tensors: *loop unrolling* to fuse iterations into a batch to preserve the locality, and *early elimination of padding* to reduce the computation load brought by padding. Specifically, we show how to apply these guidelines through JOIN where both of them occur. For simplicity, we assume that the *build* phase in hash-join has been completed, and focus only on the *probe* phase.

Loop Unrolling. Without loss of generality, assume that after the *build* phase, IDs of tuples in the build table S sharing the same hash value (i.e., forming a hash bucket) are stored contiguously in tensor ID_S , and the lengths of each hash bucket are stored in tensor len_b . Then, each tuple r in the probe table R obtains the start position and length of the corresponding hash bucket in ID_S , denoted as $start_R$ and len_R , respectively. Then, we generate all the candidate ID pairs (s_id, r_id) sharing the same hash value, and store them in two vectors C_S and C_R , respectively. Finally, we compare join keys of corresponding elements in C_S and C_R and obtain the truly matched pairs M_S and M_R . In the following, we focus on how to efficiently generate C_S and C_R .

A straightforward approach would be to iteratively extract a single s_id in the corresponding hash buckets from ID_S for each $r \in R$ simultaneously. We use $ID_R (0, 1, \dots, |R| - 1)$ to record the ID of tuples in R , and maintain a tensor idx_R to track the index of tuple ID to be extracted from ID_S in each iteration, initialized as $start_R$. In each iteration, we first extract the corresponding tuple ID from ID_S based on idx_R , denoted as $temp_S$. Then, we append $temp_S$ and ID_R to C_S and C_R respectively. Since each tuple $r \in R$ needs different numbers of iterations due to irregular hash bucket sizes, we check whether the current iteration count equals len_R and remove completed tuples from ID_R , len_R , and idx_R . Then, we increase idx_R by 1 for the next iteration. Such process repeats for $l_{\max} = \max(len_R)$ times. Note that ID_R , len_R , and idx_R are repeatedly read and updated, leading to significant overhead. Moreover, all the s_id for the same r_id are stored adjacently in ID_S but are retrieved separately across iterations, which destroys the spatial locality.

To avoid frequent materialization of intermediate results and loss of spatial locality caused by loops, we propose a *loop unrolling* strategy that transforms iterative loop structures into batched tensor operations. The main idea is to first eliminate the dependency between iterations when possible to enable parallel execution across all iterations, and then carefully combine the input

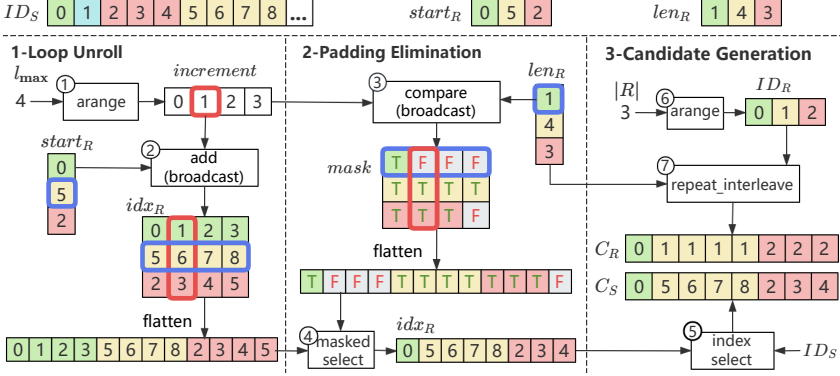


Fig. 3. Loop unrolling and padding elimination

tensors from each iteration into a unified tensor to maximize the parallelism and preserve spatial locality in the following computation. The detailed steps of *loop unrolling* to generate C_S and C_R are as follows. As shown in Alg. 1, we first use computation padding (instead of immediate deletion of finished tuples) to extend the computation for all r_id to l_{max} iterations. This ensures fixed input dimensions ($|R|$) of ID_R , len_R , and idx_R in each iteration so that we can eliminate dependency between iterations. Since idx_R increments by 1 per iteration, we pre-compute an *increment* tensor (0, 1, ..., $l_{max} - 1$) via *arange* (line 1) to record the difference of idx_R in each iteration against the initial value $start_R$. This allows us to compute each element of idx_R independently by performing broadcast addition on $start_R$ and *increment*. Specifically, we reshape $start_R$ into a column vector of $|R| \times 1$, and logically propagate *increment* to each element $start_R$. This efficiently exploits reuse of both *increment* and $start_R$ and generates the complete $|R| \times l_{max}$ index matrix idx_R in one operation (line 2). Notably, the reshape operation (view) incurs almost zero cost as it only modifies tensor metadata.

Early Elimination of Padding. The results generated by padded computation during loop unrolling should be identified and eliminated to ensure result correctness. Thus, we propose an *early padding elimination* strategy, which eliminates padding as early as possible to reduce computation overhead in the subsequent operations. We achieve this by examining whether the iteration number of each element in the index matrix idx_R is less than its corresponding hash bucket size immediately after loop unrolling. As shown in Alg. 1, we first generate a mask $mask$ of size $|R| \times l_{max}$ to denote the active status of each element in matrix idx_R . Each row in $mask$ represents the active state in all the iterations for a r_id (line 3). This is achieved by performing a *broadcast* comparison between len_R and *increment* (0, 1, ..., $l_{max} - 1$) representing each iteration number, thus exploiting locality by reusing len_R across all iterations and *increment* across all r_ids . Next, we filter invalid indices of padding computation on flattened idx_R by flattened $mask$ to avoid the generation of unnecessary candidate pairs (line 4). Then, we perform index selection on ID_S based on filtered idx_R to extract all valid s_ids in bulk, denoted as C_S (line 5). Since s_ids corresponding to the same r_id are stored adjacently in C_S , identical r_ids are also stored adjacently in C_R . We first generate the original row IDs of R (0, 1, ..., $|R| - 1$), denoted as ID_R (line 6), and then apply the *repeat_interleave* operation to repeat $ID_R[i]$ for $len_R[i]$ times to generate the entire C_R in bulk (line 7). Thus, early elimination of padded computation can filter unpromising candidate pairs as early as possible.

An Overall Example of Bridging the Gap. Fig. 3 shows the process of loop unrolling and early elimination of padding for tables S and R in Fig. 1. For simplicity, we only consider partial R containing three tuples r_1 , r_2 , and r_3 . Given ID_S , r_1 , r_2 , and r_3 have matched hash buckets starting

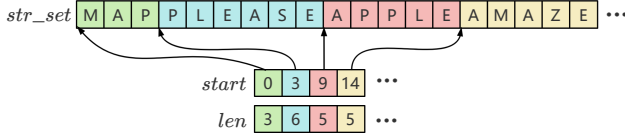


Fig. 4. Compact storage of strings

at indices 0, 5, and 2 with lengths 1, 4, and 3, respectively, as shown in $start_R$ and len_R . During loop unrolling, we first generate an *increment* vector of length 4 (the maximum bucket length) by `arange` (①), then perform broadcast addition with the transposed $start_R$ (②) to produce matrix idx_R . During padding elimination, we first perform a broadcast comparison between *increment* and len_R (③), which generates a *mask* matrix indicating that r_1 , r_2 , and r_3 have 1, 4, and 3 valid indices, respectively. Then, we filter invalid indices from the flattened idx_R based on flattened *mask* (④). These indices are then used to select the corresponding tuple IDs C_S from ID_S (⑤). For C_R , we generate the row IDs 0, 1, and 2 for r_1 , r_2 , and r_3 using `arange` (⑥), then apply `repeat_interleave` (⑦) on it based on len_R , which copies each base ID according to its bucket length (1, 4, and 3 times respectively) to obtain the final C_R .

Discussion on Skewed Data. For highly skewed data, we generate idx_R based on concatenation instead of padding. Each element in idx_R is determined by (1) the starting position $start_R[i]$ of the corresponding r_i and (2) the relative offsets in the corresponding hash bucket (0 to $len_R[i] - 1$) for all the indices of r_i . The starting positions can be easily obtained as $start_R[C_R]$, where C_R is derived directly from ID_R and len_R , as shown in Fig. 3. To compute the relative offsets, we first generate a sequential tensor *offset* ranging from 0 to $|C_R| - 1$ using the `arange` operator, then derive *prefix* through a cumulative sum of len_R , and finally subtract $prefix[C_R]$ from *offset*. Next, we generate idx_R by adding the relative offsets *offset* to their starting positions $start_R[C_R]$. The time complexity of this concatenation strategy is mainly determined by $|R| \times l_{avg}$, which remains stable regardless of the degree of skew. However, it incurs higher computational costs and more memory access to compose idx_R . Thus, we only use concatenation when the data is highly skewed, i.e., $\frac{l_{max}}{l_{avg}}$ exceeds a certain threshold.

4 Storage and Operations of Strings

We show how to store strings compactly and accelerate string operations using the optimization strategies in the guidelines.

4.1 Tensor-based String Storage and Operations

We represent a column of variable-length strings using three compact tensors to avoid storage padding: a large 1-dimensional tensor str_set to store the concatenated strings of the column, and two tensors $start$ and len to record the start indices and lengths of each string in str_set , respectively. Fig. 4 shows an example: strings of lengths 3, 6, 5, and 5 are concatenated into a long tensor with start indices 0, 3, 9, and 14, respectively. Compared with the matrix representation based on storage-padding in TQP [29], our string storage is space-efficient. While concatenating strings has also been used in existing data libraries (e.g., Arrow), manipulating the concatenated strings based on tensor operations remains challenging, as the strings vary in length while tensor operations require uniform data. Thus, we propose pre-filtering and loop unrolling strategies to efficiently map string operations into uniform tensor operations, as shown in the following.

Among string operations supported in SQL, pattern matching (i.e., LIKE) is the most fundamental and costly operation. Two special characters are used to describe patterns: `'_'` to match any character and `'%'` to match any substring. Since `'_'` can be easily handled, we only consider `'%'` in the following

Algorithm 2: start-position-aware string matching

input : a pattern $pattern$, the concatenated string set str_set along with $start$ and len , matching case $mode$

output: a mask of match result res

```

1  $str\_ID, cand\_start \leftarrow lenFilter(start, len, k, mode);$ 
2  $m' \leftarrow cand\_start.shape[0];$ 
3  $idx\_mat \leftarrow cand\_start.view(m', 1) + arange(k);$ 
4  $cand\_str \leftarrow masked\_select(str\_set, idx\_mat);$ 
5  $mask \leftarrow cand\_str == pattern;$ 
6  $mask' \leftarrow all(mask, dim = 0);$ 
7  $res \leftarrow zeros(m); res[str\_ID] \leftarrow mask';$ 
8 return  $res;$ 

```

basic cases: (1) equal with no '%', e.g., 'data' to match the same string as 'data'; (2) prefix with '%' at the end, e.g., 'data%' to match any string beginning with 'data'; (3) suffix with '%' at the beginning, e.g., '%base' to match any string ending with 'base'; (4) general with '%' at both beginning and end, e.g., '%ab%' to match any string containing 'ab'. More complex patterns, such as 'd%ab%e', can be decomposed into the above basic cases in three steps, i.e., first match 'd%' (prefix), then match '%e' (suffix), and finally match '%ab%' (general). We further classify the above cases into two categories and will discuss how to handle them respectively in the following subsections.

- *Start-position-aware matching.* The start position of comparison is fixed in the source strings, including equal and prefix, which start from the beginning of each source string, and suffix, which starts from the k -th character before the end of each source string (k is the pattern length excluding '%').
- *Start-position-unaware matching.* The start position of comparison is not fixed in the source strings, including general, which can start from any position of each source string.

4.2 Start-position-aware matching

In start-position-aware matching, for a tensor $pattern$ of length k (excluding '%' if it exists), the start position of comparison is fixed, which is the beginning (in equal and prefix) or the k -th character before the end (in suffix) of each source string. A naive solution is to compare each character in $pattern$ with all the corresponding characters in source strings from the start position simultaneously in k iterations. We optimize such matching by *loop unrolling* to extract k characters from each source string and compare them element-wise with the entire pattern simultaneously. As shown in Alg. 2, the matching process consists of three steps.

(1) Pre-filter strings and generate start positions (lines 1-2). First, we pre-filter source strings by length, i.e., remove strings with lengths not equal to k for equal or less than k for prefix and suffix, and store IDs of the remaining strings in str_ID . Then, we generate the start positions of comparison $cand_start$ for the remaining m' strings, i.e., the first (equal and prefix) or the k -th position before the end (suffix).

(2) Unroll loops by extracting candidate substrings. First, we generate an incremental tensor $0, \dots, k-1$ using *arange* operation, and add it to $cand_start$ by broadcast addition, where the incremental tensor and $cand_start$ are replicated along row and column, respectively. This generates an $m' \times k$ index matrix idx_mat where each row represents the offset array of a candidate

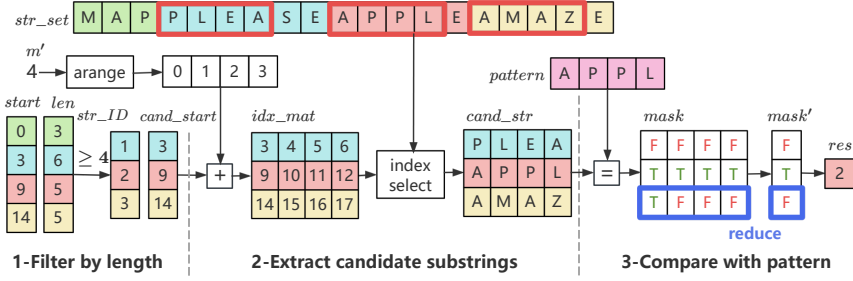


Fig. 5. Start-position-aware matching (prefix)

substring in str_set (line 3). Then, we extract all the substrings in batch from str_set based on idx_mat , and store them in $cand_str$ of size $m' \times k$, where each row represents a substring (line 4).

(3) Compare candidate substrings with the pattern. First, we compare each character in $cand_str$ with $pattern$ by broadcast comparison and obtain a bit matrix indicating whether each character is identical to the corresponding character in the pattern (line 5). Then, we reduce each row of this bit matrix through logical_and to obtain the matching result of each substring (line 6). Finally, the matching result is filled into the result tensor (line 7).

We further illustrate the above algorithm with a prefix example. As shown in Fig. 5, given a pattern "APPL" with 4 characters, we first filter the string "MAP" with length smaller than 4, then select substrings of length 4 from the remaining three strings "PLEA", "APPL", and "AMAZ", and compare them with "APPL" to obtain the matching result "APPLE".

4.3 Start-position-unaware matching

In start-position-unaware matching, for a pattern of length k , the start position of comparison can be an arbitrary position in the source string of length l . The brute-force algorithm is to extract $l - k + 1$ candidate substrings of length k from the source string with start indices from 0 to $l - k$ and then gradually compare them with the pattern and terminate when a match is found. Since adjacent substrings share common sequences of characters, different *jump* strategies have been proposed to skip unnecessary comparisons [12, 23, 37, 68, 71]. However, directly applying them to the tensor str_set containing m source strings will result in loops with irregular workload. We illustrate this by KMP [37], which exhibits more predictable and steady memory access patterns [67]. KMP jumps to the next position by maintaining a prefix table. To process the strings in str_set simultaneously, we use two vectors pos_s and pos_p of length m to record the current matching position of each source string and the pattern, which are initialized as zeros. If the character in source string s_i matches the corresponding character in pattern, increase $pos_s[i]$ and $pos_p[i]$ by 1; otherwise, look up the prefix table and update $pos_s[i]$ and $pos_p[i]$ to jump to the next comparison position. We repeat this process until we find a match or reach the end of the source string. Moreover, each string may require varying numbers of iterations, and the updates of $pos_s[i]$ and $pos_p[i]$ also diverge across strings in each iteration, leading to workload irregularity.

To avoid loops and irregularity caused by the jump strategy, we propose a loop-free strategy that converts start-position-unaware matching to start-position-aware matching through loop unrolling and irregularity elimination. The main idea is to treat the concatenated string set str_set as a whole string by ignoring the boundaries between source strings, and all the possible candidates of length k can be identified by the start positions from 0 to $N - k - 1$ and then be examined to find the true matches. This process is shown in Alg. 3, which consists of the following steps.

(1) Eliminate unpromising candidates by pre-filtering. We pre-filter unpromising substrings at the early stage by comparing the head and tail characters of all the candidate strings with

Algorithm 3: start-position-unaware string matching

input : a pattern string *pattern*, the concatenated string set *str_set* along with *start* and *len*
output : a mask of match result *res*

```

1  $h\_mask \leftarrow str\_set[0 : N - k] == pattern[0];$ 
2  $t\_mask \leftarrow str\_set[k - 1 : N] == pattern[-1];$ 
3  $ht\_mask \leftarrow \text{logical\_and}(h\_mask, t\_mask);$ 
4  $cand\_start \leftarrow \text{masked\_select}(\text{arange}(N - k + 1), ht\_mask);$ 
5  $mid\_mask \leftarrow \text{substring\_comp}(cand\_start + 1, pattern[1 : -1]);$ 
6  $cand\_start \leftarrow \text{masked\_select}(cand\_start, mid\_mask);$ 
7  $host\_ID \leftarrow \text{bucketize}(cand\_start, start);$ 
8  $valid\_mask \leftarrow (cand\_start + k) < start[host\_ID + 1];$ 
9  $host\_ID \leftarrow \text{masked\_select}(host\_ID, valid\_mask);$ 
10  $res \leftarrow \text{zeros}(m); res[host\_ID] \leftarrow \text{True};$ 
11 return res;

```

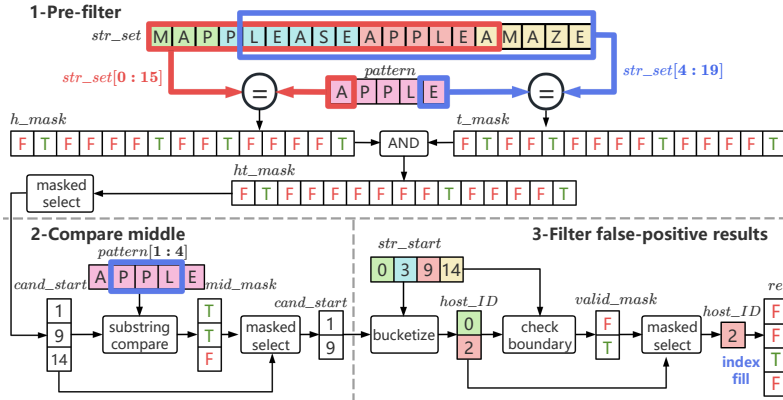


Fig. 6. Start-position-unaware matching

the corresponding characters in the pattern. Specifically, we compare $str_set[0 : N - k]$ and $str_set[k - 1 : N]$ with $pattern[0]$ and $pattern[-1]$, respectively, and obtain two masks (lines 1-2). Then, we combine these two masks by a logical AND operation and eliminate unqualified candidates via masked selection (lines 3-4).

(2) **Compare middle by start-position-aware matching.** We compare the remaining $k - 2$ middle characters of each candidate and those of the pattern by *start-position-aware matching*, as start positions ($cand_start + 1$) and lengths ($k - 2$) of these candidates are fixed. This can efficiently exploit the locality of adjacent substrings sharing common sequences of characters. Thus, we obtain *mid_mask* of size m indicating the comparison results (line 5) and only keep the start indices of qualified candidates (line 6).

(3) **Filter false-positive (FP) results by string boundaries.** Since the qualified candidates may cross the boundaries of source strings, we need to filter such false-positive matches. We first locate the host string IDs where the start indices of candidates reside, denoted as *host_ID* (line 7). Then, we check whether the end index of each candidate is less than the end index of its host string to generate the valid host string IDs (lines 8-9). Finally, we assign True to the valid host strings and return the final results (lines 10-11).

Algorithm 4: Index join

input : join keys key_S and key_R of tables S and R
output : match pairs M_S and M_R

- 1 $sorted_keys, sorted_ID \leftarrow \text{sort}(key_S)$;
- 2 $index, counts \leftarrow \text{unique_consecutive}(sorted_keys)$;
- 3 $prefix \leftarrow \text{cumsum}(counts) - counts$;
- 4 $ID_R, bucket_{ID} \leftarrow \text{probe}(key_R, index)$;
- 5 $ID_S, start_R, len_R \leftarrow sorted_ID, prefix[bucket_{ID}], counts[bucket_{ID}]$;
- 6 $M_S, M_R \leftarrow \text{gen_match_pair}(ID_S, ID_R, start_R, len_R)$; // Alg. 1
- 7 **return** M_S, M_R ;

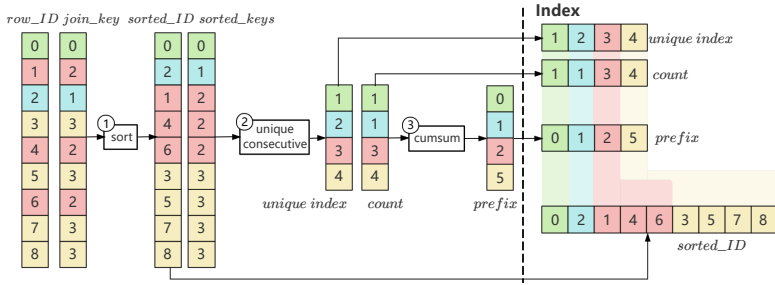


Fig. 7. Tensor-based index join

Fig. 6 provides an example to illustrate the above process. For pattern "APPLE" with length 5, all the 15 candidates of length 5 can be recorded by the start positions from 0 to 14. We then pre-filter strings by comparing the first and the last characters of each candidate (accessed via $str_set[0 : 15]$ and $str_set[4 : 19]$, respectively) against the pattern's first character "A" and last character "E", and obtain 3 candidates starting at positions [1, 9, 14]. Next, we verify the middle part "PPL" of the pattern against the remaining candidates using start-position-aware matching, which further narrows the results to candidates starting at [1, 9]. In the final stage, we identify the host string IDs [0, 2] corresponding to candidates [1, 9], and eliminate the candidate starting at position 1 as a false-positive since it crosses the boundary of its host string. Only the candidate starting at position 9 remains valid, indicating a successful match for its source string (string with ID 2).

5 Tensor-based SQL Operators

We first discuss how to efficiently map the most common and costly SQL operators JOIN and AGGREGATE to tensor programs, and then extend them to multi-XPUs (e.g., GPUs, TPUs).

5.1 Tensor-based Join

We first propose the index join and improved hash join algorithms for inner join, and then discuss extensions to outer-/semi-join. When join tables differ significantly in size, we use the smaller table S to build index or hash table, and use the larger table R to look up or probe. For tables of similar size, we can: (1) choose the table with unique/sorted join column as S to simplify the build process; (2) choose the targeted table as R for semi-/outer-join (e.g., choose A for $A \times B$) to efficiently obtain matched/unmatched row IDs; (3) otherwise, randomly select a table as S .

Index Join. Index join shown in Alg. 4 consists of three phases: (1) construct an index for table S (lines 1-3); (2) look up the index using tuples in table R (line 4); (3) generate match pairs (lines 5-6). We also give an example in Fig. 7 to illustrate the index construction. Specifically, we sort the join

key of S to obtain *sorted_keys* as well as the permuted s_id correspondingly to obtain *sorted_ID* (①, line 1). Then, we remove all but the first one of consecutive duplicate keys in *sorted_keys* by *unique_consecutive* to obtain a unique index and a tensor *count* to record the occurrence of each unique key (②, line 2). Each element in the unique index is associated with a list of adjacent s_id in *sorted_ID*, denoted as a join bucket. We then generate *prefix* by cumulative sum on *count* to locate the start position of each bucket in *sorted_ID* (③, line 3). During the lookup phase, each tuple r_i of the large table R looks up the unique index using binary search to find the r_id and corresponding join bucket ID if matched, which are stored in ID_R and $bucket_{ID}$, respectively (line 4). Finally, during the match pair generation phase, each matched tuple r_i first examines *prefix* and *count* to identify the start position and the length of its corresponding join bucket (line 5), and then all matched tuples extract their join buckets simultaneously according to Alg. 1 to generate the match pairs M_S and M_R (line 6).

When the index size for S is sufficiently small to reside in cache, the binary search is cache-friendly. However, when the index size exceeds the cache capacity, the binary search incurs more frequent cache misses. Thus, we further sort R , which introduces additional sorting overhead but improves data locality during binary search.

Improved Hash Join. We propose an improved hash-join algorithm, which can avoid repeated access of the *build* and *probe* tables in b_{max} iterations in TQP [29]. Our algorithm can build a hash table efficiently by accessing the build table only once and perform lookup of the hash table for all the tuples in the probe table simultaneously. To achieve this, we propose two optimization strategies: *compact hash table* and *bucket dependency elimination*. Since we have introduced the *probe* phase in Section 3, here we focus on how to *build* the hash table.

As stated before, storing a hash table in a matrix by padding each hash bucket will cause significant memory overhead due to the skewed distribution of the real-world data. Thus, we propose *compact hash table* to store the entries of each hash bucket consecutively and concatenate them into a 1-dimensional tensor of fixed length n (length of the build table).

However, concurrently inserting all rows into the hash table remains challenging, since the rows sharing the same hash value need to be written into a shared consecutive region of the hash table synchronously. Thus, we propose *bucket dependency elimination* to assign each row an inner-bucket offset, thus eliminating dependency between rows belonging to the same hash bucket and allowing them to perform independent insertion simultaneously. Specifically, in each bucket i , each row is assigned a unique inner-bucket offset ranging from 0 to b_i , where b_i is the size of the bucket. To generate inner-bucket offsets, we build a tensor *count* with length h initialized as all zeros, and then perform *scatter_add_with_old* for each row, in which a row with hash value i increments *count*[i] by 1 and obtains the old value before update to serve as its inner-bucket offset. This ensures that all rows sharing the same hash value i obtain distinct inner-bucket offsets ranging from 0 to b_i . Then, *count* records the number of occurrences of each hash value. This process is similar to the *scatter_add* tensor operation, but is slightly modified by keeping old values during each concurrent update. With the hash bucket IDs i and inner-bucket offset j for each row, we can directly compute the insertion destination in the compact hash table in parallel as $prefix[i] + j$, where *prefix* is the starting position of each hash bucket.

Extension to Outer-/Semi-join. We can also handle outer-/semi-join by modifying the process of generating match pairs for inner join. Recall that we have obtained the matched row IDs ID_R in R and the corresponding join bucket IDs $bucket_{ID}$ (line 5 in Alg. 4). For semi-joins, we identify unique matched row IDs in either table S or R . For right semi-join $S \bowtie R$, we directly return ID_R as the unique matched row IDs in R . For left semi-join $S \ltimes R$, since multiple $r \in R$ may match the same bucket, we extract unique values from $bucket_{ID}$ and retrieve the row IDs in S for each

Algorithm 5: Improved Group Aggregation

input : group column *key*, aggregate column *cdata*, and the aggregate function *aggfunc*
output: aggregated result *agg_res*

```

1 sorted_keys, sorted_ID  $\leftarrow$  sort(key);
2 cdata  $\leftarrow$  cdata[sorted_ID];
3 group_ID, counts  $\leftarrow$  unique_consecutive(sorted_keys);
4 g  $\leftarrow$  counts.shape[0]; n  $\leftarrow$  cdata.shape[0];
5 v  $\leftarrow$  calculate_v(g, n);
6 vagg_res  $\leftarrow$  zeros(g  $\times$  v);
7 vgroup_ID  $\leftarrow$  group_ID  $\times$  v + randint(low = 0, high = v, size = n);
8 vagg_res.scatter_reduce_(vgroup_ID, cdata, aggfunc);
9 agg_res  $\leftarrow$  vagg_res.view(g, v).sum(dim = 1);
10 return agg_res;

```

unique bucket. For outer-joins, we identify unmatched rows in table *S* or *R*, pair them with NULL values, and concatenate them with the results generated by inner join. For right outer-join $S \bowtie R$, we derive the unmatched rows in *R* from the matched row IDs ID_R . For left outer-join $S \bowtie R$, we generate the unmatched bucket IDs similarly to the left semi-join and obtain the corresponding unmatched row IDs in *S*.

5.2 Tensor-based Group Aggregate

The SQL operator AGGREGATE divides tuples with the same value on grouping attributes into groups, and aggregates each group with basic (e.g., SUM, MIN, MAX, AVG, COUNT) and advanced (e.g., MEDIAN) aggregate functions with optional clauses (e.g., DISTINCT).

Aggregate with Basic Functions. As shown in Alg. 5, a tensor-based aggregate algorithm generally contains the following basic steps. First, we sort grouping attributes *key* to obtain *sorted_key* and permute related aggregate columns *cdata* accordingly (lines 1-2). Then, we apply unique_consecutive on *sorted_key* to remove duplicate values, producing l_g groups and generating the corresponding group IDs (ranging from 0 to $l_g - 1$) for each row (denoted as *group_ID*) along with the size of each group (denoted as *counts*) (line 3). For aggregate function COUNT, we can directly return *counts* and stop; for other aggregate functions, we will proceed to the following aggregation phase. To perform aggregation for all the l_g groups simultaneously, we build a global result tensor *agg_res* with length l_g . Then, we can aggregate the permuted *cdata* to corresponding groups based on *group_ID* by the *scatter_reduce* operator, i.e., each tuple with group ID *i* updates the *agg_res*[*i*] in parallel. Nevertheless, tuples with identical group IDs are stored adjacently after sorting, which increases the risk of concurrent writes to *agg_res*[*i*] during parallel reduction, resulting in severe memory contention, especially for large groups. These contended updates to *agg_res*[*i*] must be serialized, and thus diminish the parallelism of hardware accelerators [26]. To mitigate memory contention, one possible approach is to shuffle the tuples so that the atomic updates to the same group can be executed at different time intervals. However, shuffling is not only costly, but also destroys the locality of sorted group ID, which significantly increases cache misses and ultimately increases memory overhead.

Thus, we propose *group extension* to alleviate the contention issue while preserving the computational locality (lines 4-9). Specifically, we subdivide each group into *v* virtual subgroups (*v* is determined based on the current row number *n* and group number *g*), resulting in a $g \times v$ matrix,

denoted as *vagg_res* (lines 4-6). Contiguous rows within the same group are distributed across different subgroups to alleviate atomic contention while keeping computational locality. Given *vagg_res*, we first randomly generate subgroup IDs ranging from 0 to $v - 1$ for each row. A row with group ID i and subgroup ID j is then mapped and reduced to *vagg_res*[i, j] (lines 7-8). Finally, we perform a reduction on the rows of *vagg_res* to obtain the aggregated result (line 9). We need to carefully select v to optimize the overall performance. If v is too small, the memory contention cannot be well alleviated; if v is too large, the second aggregation will cost more time. Our experiments indicate that when the average group size $\frac{n}{g}$ is larger than 100, v can be directly set to $\lceil \frac{n}{g \times f} \rceil$ where f is a tuned parameter from statistics, and v is bounded by a maximum threshold $v_{\max}$.

Aggregate with Advanced Functions and Clauses. For advanced functions (MEDIAN) and optional clauses (DISTINCT), we process all the data across all groups in a batch by encoding *cdata*. Take COUNT DISTINCT as an example. We encode *cdata* by $encode = group_{ID} \times val_{\max} + cdata$, where $val_{\max} = \max(cdata) + 1$. Then we sort *encode* to ensure that data in the same group is stored contiguously in ascending order, and remove the duplicate values within each group in *encode* by unique_consecutive to obtain *encode'*. The corresponding group IDs are recovered from *encode'* by $group'_{ID} = \lfloor \frac{encode'}{val_{\max}} \rfloor$, and then used to count the occurrence of each group ID to obtain the COUNT DISTINCT result. For MEDIAN, the data are encoded and sorted similarly, and we can obtain column data sorted and organized by groups from *encode* by $cdata' = \text{mod}(encode, val_{\max})$. We then find indices of the medians of all the groups in *cdata'* as follows: obtain group size *counts* by unique_consecutive on *group_{ID}*; generate *prefix* by applying cumulative sum on *counts* to locate the start position of each group; obtain the indices of median by $prefix + \lfloor \frac{counts+1}{2} \rfloor$.

5.3 Extension to Multi-XPUs

We also extend our system to multi-XPUs, since the memory capacity of a single XPU is usually too limited to support large-scale datasets. We directly store tables on XPUs to enable parallel processing, and leverage the high bandwidth direct links between XPUs to facilitate potential data exchange during execution. We mainly focus on two key aspects: how to design a load-balanced partition strategy to maximize XPU utilization while minimizing data exchange overhead [46, 85]; how to support join and aggregate operators that involve data exchange (other simple operators can be easily distributed).

Balanced Data Partition. For tables with a significant size disparity, we mainly focus on evenly distributing large tables to achieve load balance. For tables with similar sizes, we can partition tables that are frequently joined to reduce data exchange. Thus, we propose a *balanced hash-partition* strategy to achieve load balance while minimizing data exchange. Specifically, we first hash large tables based on their most frequently used join key. Then, we divide the range of hash values into n_g contiguous segments, ensuring each segment contains a similar number of tuples, where n_g is the number of XPUs. The remaining tables are partitioned to the corresponding XPUs based on the same hash function, allowing the majority of expensive joins to be evaluated locally. The metadata of the partitioned tables, such as the partition column and hash range for each XPU, are recorded to facilitate data exchange. To further reduce data exchange, we can also broadcast small tables on all the XPUs, e.g., the dimensional tables in the star schema.

Join on Multi-XPUs. Given the join condition, if both join tables are broadcast or partitioned according to the join key, join can be evaluated locally, referred to local join. Otherwise, we need to exchange/repartition data across XPUs. First, we check whether the larger join table is partitioned based on the join keys. If not, we repartition the required columns based on our *balanced hash partition* strategy to the corresponding XPUs. Then, the smaller tables are co-partitioned to each XPU based on the same hash function to enable local join. Considering the limited transfer

Table 2. Query time (s) comparison on TPC-H

Query	SF = 1									SF = 100		
	64 cores			1 × P100		1 × V100				64 cores	4 × V100	
	DuckDB	TQP	TQEx	TQP	TQEx	Blazingsql	HeavyDB	TQP	TQEx	DuckDB	HeavyDB	TQEx
Q1	0.035	14.623	0.445	0.109	0.010	0.202	0.032	0.056	0.007	1.457	0.071	0.099
Q2	0.050	0.982	0.040	0.019	0.014	0.333	0.133	0.028	0.013	0.318	1.668	0.104
Q3	0.063	0.456	0.269	0.012	0.007	0.151	0.052	0.013	0.006	1.677	N/A	0.057
Q4	0.053	0.459	0.100	0.009	0.004	0.103	0.295	0.009	0.003	1.267	N/A	0.032
Q5	0.050	0.252	0.046	0.018	0.009	0.172	0.029	0.019	0.008	1.042	4.621	0.047
Q6	0.016	0.092	0.120	0.001	0.001	0.046	0.014	0.001	0.001	0.450	0.026	0.017
Q7	0.107	0.411	0.300	0.018	0.011	0.264	0.028	0.021	0.009	5.124	2.941	0.072
Q8	0.061	0.247	0.074	0.022	0.010	0.278	0.044	0.024	0.008	1.348	4.717	0.060
Q9	0.100	1.668	0.346	0.067	0.017	0.357	0.029	0.080	0.022	3.694	6.267	0.425
Q10	0.074	0.830	0.260	0.014	0.009	0.220	0.028	0.016	0.009	1.505	6.756	0.172
Q11	0.017	0.213	0.006	0.010	0.005	N/A	0.054	0.012	0.005	0.131	0.690	0.018
Q12	0.049	0.266	0.077	0.015	0.007	0.089	0.025	0.011	0.005	1.182	2.845	0.057
Q13	0.056	6.580	0.282	0.192	0.017	0.128	0.039	0.230	0.034	2.198	4.493	0.336
Q14	0.042	0.145	0.164	0.004	0.003	0.083	N/A	0.004	0.002	1.706	N/A	0.038
Q15	0.042	0.980	0.186	0.010	0.005	0.127	0.135	0.007	0.004	1.108	0.164	0.077
Q16	0.052	0.723	0.080	1.573	0.012	N/A	3.913	1.553	0.037	0.411	9.810	0.094
Q17	0.046	21.760	0.186	0.017	0.011	0.125	0.127	0.015	0.008	0.401	3.135	0.376
Q18	0.091	25.058	0.587	0.038	0.017	0.187	0.516	0.031	0.013	8.647	N/A	0.412
Q19	0.047	0.086	0.246	0.022	0.015	N/A	0.022	0.014	0.007	0.990	0.168	0.179
Q20	0.041	4.674	0.240	0.015	0.009	0.175	0.222	0.017	0.007	1.028	N/A	0.343
Q21	0.131	0.327	0.357	0.031	0.021	N/A	N/A	0.030	0.017	3.388	N/A	0.215
Q22	0.039	0.053	0.021	0.006	0.004	N/A	N/A	0.008	0.004	0.650	N/A	0.031

bandwidth and computation overhead, we choose to broadcast the smaller table to avoid large table exchange when necessary.

Aggregate on Multi-XPUs. For the aggregate operator, if the table to be aggregated is partitioned based on the aggregation keys, all elements belonging to the same aggregation key are on the same XPU. Thus, aggregate can be evaluated locally. Otherwise, we need to exchange data. To reduce the amount of data to be exchanged, we perform aggregation in three steps. First, we perform local aggregation on each XPU; second, we exchange the locally aggregated results; third, we perform the final aggregation on each XPU to obtain the overall result.

6 Experiments

We evaluate the performance of our system TQEx through comparison studies with state-of-the-art baselines and ablation studies on our optimization techniques.

6.1 Experimental Setup

Benchmark Setup. We use TPC-H [22] with 22 queries as the benchmark, and generate data at different scale factors (SF) (from 1 to 300, where 1 approximates 1GB of total data) using dbgen. Data is loaded into the Pandas DataFrame from disk and converted into the internal format of each system, e.g., CUDA DataFrame for BlazingSQL/HeavyDB, and PyTorch tensors for TQP and TQEx. After that, the data is moved to the GPU for computation. During data updates, we update the affected columns directly in the device memory according to the update statements. Each query runs 10 times, where the first 5 runs act as a warm-up and the average of the last 5 runs is reported.

Baseline Systems. We compare our system with multiple state-of-the-art query processing systems for different hardware settings. Specifically, we compare our system with CPU-based DuckDB

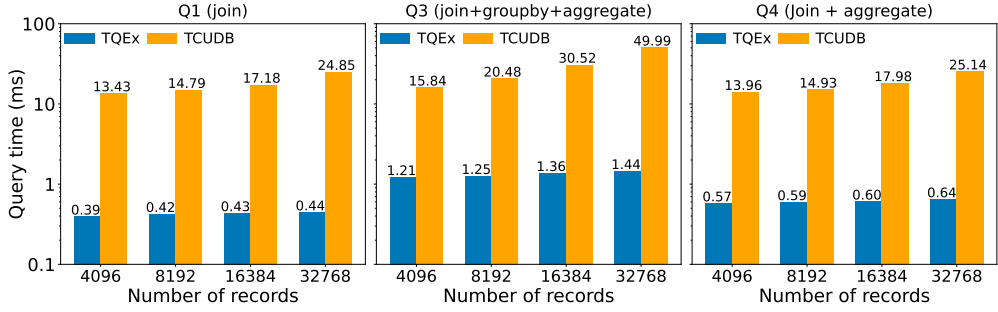


Fig. 8. Query time comparison between TCUDb and TQEx

[59], GPU-based HeavyDB [2], BlazingSQL [1], and tensor-based TQP [29], which can support TPC-H benchmark. We also compare our system with TCUDb[31] based on Tensor Core Units on its tailored queries, as it cannot support TPC-H queries. We implemented TQEx and TQP, and obtained the source code of other baselines.

Hardware and Software Setup. For CPU experiments, we use AMD EPYC 7742 with 64 cores, 128 threads, and 512GB RAM. For single-XPU experiments, we use NVIDIA Tesla V100 by default. For multi-XPU experiments, we use NVIDIA 4 × V100 GPUs and NVIDIA 4 × A800 GPUs. Other configurations are PyTorch 2.2.1, CUDA 12.1, DuckDB 1.1.1, HeavyDB 8.0.1, and BlazingSQL 21.08.

6.2 Overall Performance

We evaluated the overall performance of DuckDB, HeavyDB, BlazingSQL, TQP, and TQEx on TPC-H at SF 1 and 100, respectively, as shown in Table 2. For the CPU system DuckDB, we always run it on a 64-core CPU to achieve the best performance. For GPU-based systems, we run them on a single V100 GPU for SF 1 and 4 V100 GPUs for SF 100, respectively. We also evaluated TQP and TQEx on P100, the back-end used in [29], demonstrating that our implementation of TQP has competitive performance with the original paper [29]. Note that we also evaluate TQP and TQEx on the 64-core CPU to show their portability on CPU but not performance gain due to the less optimization of TCRs on CPU. For SF 1, DuckDB, TQP, and TQEx can support all the 22 queries while BlazingSQL and HeavyDB only successfully answered 17 and 19 queries, respectively. Overall, GPU systems outperform CPU systems. The CPU-based TQP and TQEx do not perform as well as DuckDB on most queries, which is reasonable as the underlying TCRs are not optimized for CPU. However, their GPU-based versions are faster than DuckDB by an order of magnitude for most queries. Particularly, TQEx on GPU is 5.5× faster than DuckDB, which shows a significant improvement when using GPU. Among the GPU systems, TQEx significantly outperforms BlazingSQL and HeavyDB, and achieves a speedup of 9.6× speedup (with a peak of 41.9×) over TQP. For SF 100, we evaluate HeavyDB and TQEx, as BlazingSQL and TQP cannot directly support multi-GPUs. TQEx is much faster than DuckDB and HeavyDB. For DuckDB, TQEx achieves speedups even up to 20× for 10 out of 22 queries. The total query time of TQEx is about 12.2× faster than DuckDB. Specifically, TQEx even achieves a speedup of 71.1× compared to DuckDB on Q7. Moreover, TQEx outperforms HeavyDB on 13 out of 15 queries supported by HeavyDB. On average, TQEx is 22.7 × faster than HeavyDB, where the speedup is nearly two orders of magnitude on Q5 and Q8.

We also compare TQEx with TCUDb [31] on the three queries (Q1, Q3, Q4) evaluated in their original paper, since TCUDb cannot support TPC-H due to the lack of required SQL operators such as MIN, MAX, CASE, and LIKE. Specifically, Q1 involves join; Q3 includes join, group-by, and aggregate; Q4 consists of join and aggregate. As shown in Fig. 8, when the number of records in the tables increases (both participating tables have the same size and 32 distinct values by default

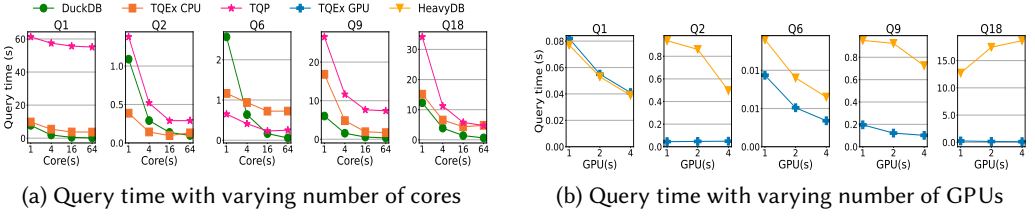


Fig. 9. Scalability on varying number of cores/GPUs

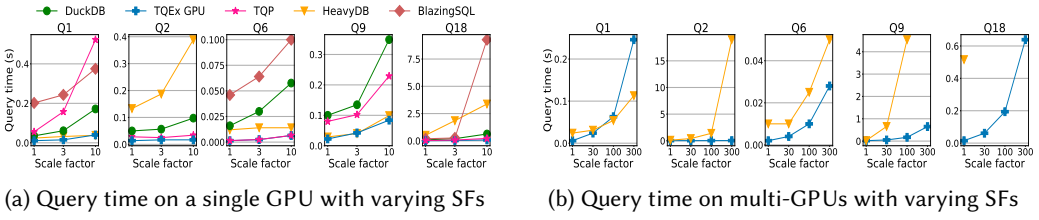


Fig. 10. Scalability on data with varying SFs

[31]), the query time of TQEx and TCUDB slowly increases, and TQEx significantly outperforms TCUDB on all three queries, achieving speedups ranging from 13 $\times$ to 56 $\times$.

6.3 Scalability

Following TQP [29], we select 5 representative queries and test them under different numbers of cores/XPUs and scale factors to comprehensively evaluate the systems.

Scaling the Number of Cores/GPUs. We evaluate the systems on different numbers of CPU cores and GPUs, respectively. Fig. 9a shows the query performance at SF 10 as the number of CPU cores increases from 1 to 64. DuckDB shows good scalability across these 5 queries, while the improvement of TQEx and TQP is not as significant as that of DuckDB. The underlying reason is that tensor operations in PyTorch have not been well optimized for multi-core CPUs. Fig. 9b shows the effect of the number of V100 GPUs on query performance at SF 30. As the number of GPUs increases, the execution time of most queries decreases. For aggregate queries Q1 and Q6 on single tables, the query time of both TQEx and HeavyDB decreases significantly when the number of GPUs increases, as the workload of aggregation can be easily distributed across multiple GPUs for speedup. For queries Q2, Q9, and Q18 involving join, the decrement of the query time is limited as the exchanged data also increases with the increase in the number of GPUs. The query time of Q18 in HeavyDB even increases, implying that HeavyDB has not been well optimized for complex queries involving both join and aggregate for multi-GPUs.

Scaling the Data. We evaluated the systems at different SFs on a single GPU and multiple GPUs, respectively. Fig. 10a shows the query time with SFs from 1 to 10 on a single V100 GPU. TQEx demonstrates excellent scalability across 5 query tasks, while TQP scales poorly on Q1, HeavyDB scales poorly on Q2 and Q18, DuckDB scales poorly on Q6 and Q9, and BlazingSQL scales poorly across all the queries. In addition, BlazingSQL only supports Q2 and Q9 at SF 1. Our system TQEx outperforms HeavyDB, BlazingSQL, and TQP for all the queries, demonstrating efficient GPU resource utilization for complex queries on large-scale data. Fig. 10b shows the query time of evaluated systems when scaling the dataset from 1GB to 300GB on 4 $\times$ A800. Here, we only show TQEx and HeavyDB that can run on multiple GPUs. As SF increases, TQEx can scale well on all the queries, especially for costly Q2 and Q9 involving join operators. In contrast, HeavyDB scales

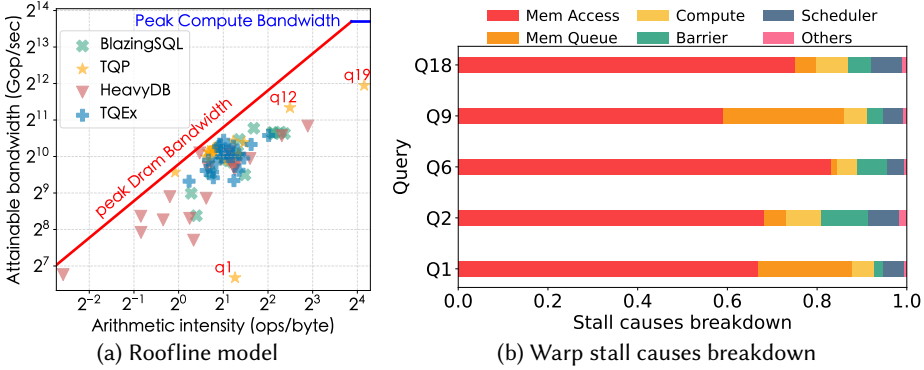


Fig. 11. Bottleneck analysis

poorly on queries involving joins (Q2, Q9, Q18). Especially for Q18, HeavyDB can only deal with a small dataset with SF less than 30.

6.4 Bottleneck Analysis

We conducted bottleneck analysis based on the well-known roofline model and the detailed warp stall breakdown following the analysis of the GPU database systems in [14]. We use Nsight Compute [49] to collect the metrics listed in [14].

Roofline Model Analysis. We first test whether the systems are memory-bound or compute-bound by the roofline model [79]. As shown in Fig. 11a, the model contains two lines indicating the peak memory bandwidth (α) and peak compute bandwidth (β) of V100 as the performance ceiling. For each system, we report the attainable bandwidth (i.e., throughput), calculated as operations per second on the Y-axis against the arithmetic intensity (AI) on the X-axis, defined as the number of operations divided by the number of bytes accessed. Here, we record the integer operations as most queries require only integer operations. We can see that all the queries in TQEx are memory-bound with AI below $\frac{\alpha}{\beta}$, and attain stable bandwidth near the ceiling. Most queries in TQP are also memory-bound except Q19, which contains intensive string operations over the intermediate result of join, while we can reduce such computation by our optimized string operations. The throughput of Q1 in TQP even drops to 4.65% due to the severe memory contention during aggregation, while TQEx alleviates this by group extension, achieving 74.3% throughput. BlazingSQL and HeavyDB are also not as stable as TQEx.

Warp Stall Analysis. We further evaluate the warp execution efficiency to provide microstructure-level insights. Across all queries, TQEx maintains 9.90 active warps per scheduler cycle but issues only 0.19 warps per scheduler cycle. This indicates schedulers remain idle 80% of cycles. We further categorize the stall causes of warps in Fig. 11b, showing that over 70% of stalls across queries are memory-associated (memory access or memory queue full), consistently validating the memory-bound nature of our systems.

6.5 Cost/Performance Trade-off

We conduct a cost/performance analysis by comparing our GPU-based system TQEx with the CPU-only baseline DuckDB. The prices on Azure servers are used as a reference. Specifically, we select a CPU-only machine D32_v5 with 32 cores and 128GB of memory. Then, we compare the DuckDB on D32_v5 with our system on (1) NC6s_v2 with an NVIDIA P100 GPU, about 50% more expensive than D32_v5, (2) NC6s_v3 with an NVIDIA V100 GPU, about 63% more expensive than D32_v5, and (3) NC24ads_A100_v4 with an NVIDIA A100 GPU, about 1.7× more expensive

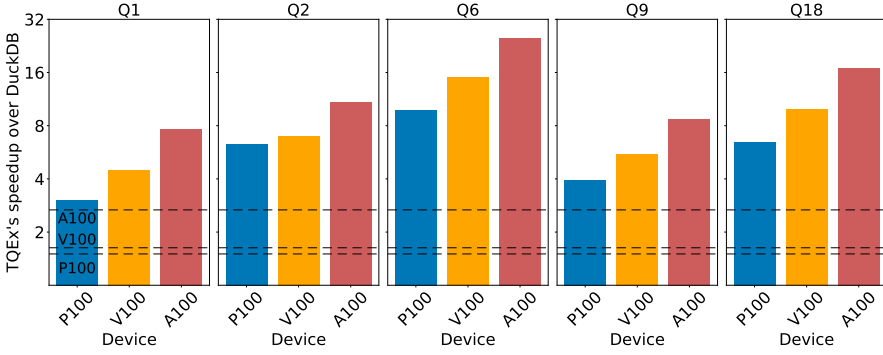


Fig. 12. Cost/performance trade-off

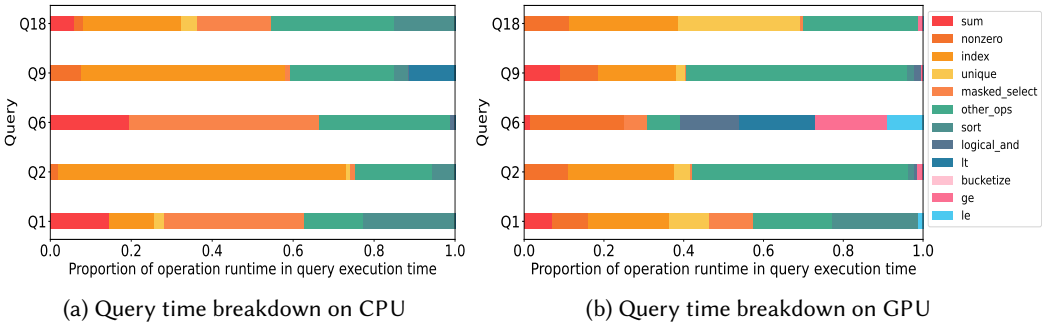


Fig. 13. Query time breakdown for tensor operations

than D32_v5. For each GPU type, we evaluate the relationship between the speedup of TQEx over DuckDB and the price ratio, as shown in Fig. 12. The dashed lines represent the price ratio between the GPUs and the 32-core CPU, while the bar chart depicts the speedup of GPUs relative to the 32-core CPU. The results show that the height of the bars for all queries significantly exceeds their respective dashed lines, which indicates that TQEx not only performs well in terms of query time but also outperforms DuckDB in terms of cost-effectiveness.

6.6 Performance Breakdown

We delve into the performance of our system by using profiling tools [4] to analyze which tensor operations contribute a significant part to the query time. Fig. 13a and 13b show the query time breakdown of the representative queries on CPUs and GPUs, respectively. Even if we use the same algorithms on both CPU and GPU, the same operator shows different contributions for the same query. For example, Q6 spent most of the time on *masked_select* on CPU, while on GPU, most is spent on *nonzero* and comparison operators such as *lt*, *gt*, *le*; for Q18, the *unique* operator takes up only 4% of the time on CPU but increases to 30.7% on GPU. Across queries, the majority of time is also spent on different operators for different hardware. On CPU, most queries are bounded by *index* or *masked_select*, while on GPU, most queries are bounded by *nonzero*, *unique*, or comparison operators. This is because the operator implementations are different for CPU and GPU in TCRs.

6.7 Portability

We also deployed TQEx and TQP across different hardware backends to evaluate their portability. Besides V100, we evaluated them on two desktop machines with distinct hardware configurations:

Table 3. Query time (s) over different hardware backends

Query	Method	NVIDIA V100	AMD 7700XT	Apple M2 Pro
Q2	TQEx	0.013	0.003	0.027
	TQP	0.028	0.007	0.057
Q6	TQEx	0.001	0.001	0.005
	TQP	0.014	0.001	0.006
Q9	TQEx	0.022	0.011	0.255
	TQP	0.080	0.051	0.782
Q18	TQEx	0.013	0.017	0.083
	TQP	0.031	0.045	0.978

Table 4. Query time (s) of different versions of TQEx

Query	TQEb	TQEs	TQej	TQEa	TQEx
Q1	0.648	0.650	0.650	0.037	0.036
Q2	0.032	0.032	0.015	0.032	0.014
Q3	0.050	0.052	0.021	0.052	0.020
Q4	0.042	0.041	0.010	0.042	0.010
Q6	0.005	0.005	0.005	0.005	0.005
Q9	0.238	0.167	0.150	0.235	0.064
Q11	0.017	0.018	0.006	0.018	0.006
Q13	0.790	0.092	0.788	0.779	0.075
Q16	2.680	2.338	2.677	0.139	0.014
Q18	0.233	0.233	0.094	0.226	0.085

Machine 1 is equipped with an AMD 7700XT GPU, an Intel 12400KF CPU, and 16GB of memory; Machine 2 is a Mac Pro featuring an Apple M2 Pro chip and 16GB of memory. We evaluated four representative queries at SF 1. Table 3 shows that TQEx runs successfully across heterogeneous hardware platforms, and consistently outperforms TQP across all tested platforms, further validating its efficiency and portability. Moreover, both TQEx and TQP can achieve better performance on AMD 7700XT for most queries, which shows the necessity to build a query engine that can exploit the parallelism of heterogeneous hardware backends.

6.8 Ablation Study

We first evaluate the following version of TQEx on representative queries: the basic version TQEb, three partially-optimized versions, TQEa, TQej, and TQEs, which only adopt optimized aggregation, optimized join, and optimized string operations, respectively, and the fully optimized version TQEx. As shown in Table 4, for Q1 and Q16, the aggregate optimization TQEa plays a significant role, achieving acceleration ratios of approximately 17.6× and 19.3×, respectively, compared to the basic version. For Q13, the string optimization TQEs was effective, significantly improving performance by about 8.6×. For Q3 and Q4, the join optimization TQej was effective, with acceleration ratios of around 2.4× and 4.2×, respectively. For some queries, multiple optimizations worked together. For example, aggregate and string optimizations both significantly contribute to Q16; both join and aggregate optimizations improve the performance on Q18; all three optimizations contribute to the performance improvement on Q9.

We further investigate our string optimizations by evaluating LIKE against those in TQP [29] and cuDF [48] (state-of-the-art GPU DataFrame library) on column O_COMMENT of table ORDERS in

Table 5. Statistics of TPC-H and Wikipedia datasets

Dataset	Subset	Number of Rows	Max. Length	Avg. Length
TPC-H	ORDERS	1500000	78	48.45
Wikipedia	labels	16904466	255	20.67
	short_abstracts	6159995	1902	314.46
	long_abstracts	6159995	89069	416.64

Table 6. Query time (s) comparison for string operator LIKE

System	ORDERS		labels				short_abstracts				long_abstracts			
	SP	CP	k=4	k=6	k=8	CP	k=4	k=6	k=8	CP	k=4	k=6	k=8	CP
TQP	0.576	0.616	2.019	2.029	2.042	2.094	-	-	-	-	-	-	-	-
CUDF	0.038	0.119	0.013	0.056	0.077	0.103	0.072	0.099	0.101	0.214	0.114	0.155	0.159	0.415
TQEx	0.024	0.033	0.009	0.009	0.010	0.011	0.049	0.050	0.050	0.083	0.070	0.070	0.071	0.132

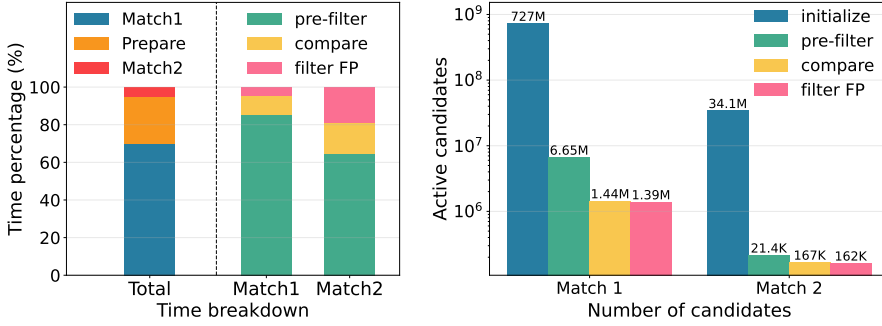


Fig. 14. Detailed evaluation of string operator LIKE

TPC-H (SF=10) [22] and three subsets (labels, short_abstracts, and long_abstracts) of Wikipedia [78] extracted from DBpedia [24]. Table 5 shows the dataset statistics. Table 6 shows the query time for simple patterns (SP) with one word '%[word]%' and complex patterns (CP) with two words '%[word1]%' '[word2]%' . For SP, we evaluate keywords "special", "pending", "unusual", and "express" of Q13 in TPC-H and five randomly selected keywords for different pattern lengths k in each subset of Wikipedia. For CP, we evaluate all possible word pairs from Q13 in TPC-H and 5 randomly selected word pairs with high frequency in each subset of Wikipedia. TQP fails on short_abstracts and long_abstracts as the excessive storage padding caused by skewness exceeds GPU memory capacity. TQEx significantly outperforms TQP by orders of magnitude, and achieves 2.3 $\times$ and 3.3 $\times$ speedup over cuDF for SP and CP, respectively.

We further profile each step in LIKE by matching O_COMMENT against the pattern '%special%deposits%'. As shown in Fig. 14, the total time 34.04ms breaks down into: (1) match '%special%' (70.2%), (2) prepare data (24.9%), (3) match '%deposits%' (4.9%). The three steps in each match occupy 85.2%/64.8%, 10.1%/16.4%, 4.7%/18.8%, respectively. We also show the number of candidates in each step. In the first match, pre-filter removes 99.1% of candidates, compare-middle further removes 78.4%, and only 3.4% of the remaining ones are invalid and are removed during false-positive filtering. The second match shows a similar trend.

7 Related works

Hardware-accelerated Databases. Integrating advanced hardware accelerators into database systems has been an active line of research over the past decades. Some earlier works using

GPUs with high computational power and memory bandwidth as co-processors for analytical query processing [13, 19, 27, 30, 41, 43, 52, 61, 76, 82, 83], where data is stored primarily on CPU and moved to GPU at query execution time. To avoid frequent data transfers between CPU and GPU, some recent work started to directly store all or a significant fraction of data on GPU [1–3, 10, 14, 64, 65]. HeavyDB [2] is a widely used commercial GPU system, which internally leverages LLVM for more flexible and optimized query execution. BlazingSQL [1] uses Thrust and cuDF as its backends for query execution. Kinetica [3] is built for real-time analytics at scale by leveraging vectorized memory-first architecture and a highly-distributed, lockless mechanism. Crystal [64] is specially optimized for the Star Schema Benchmark (SSB), which adopts a tile-based execution model to optimize memory and computational resource usage. Shanbhag et al. [65] further extend Crystal for in-GPU compression. GaccO [10] splits query processing across the CPU and the GPU. Cao et al. [14] conducted a cross-stack performance and resource-utilization analysis of SOTA GPU systems, providing optimizations for system implementation and resource allocation. Some recent works optimized specific database operators on GPUs, e.g., a perfect hashing scheme GPH to accelerate join [15], and the gather-from-transformed-relations (GFTR) strategy [80] to accelerate join and aggregate. Other works also explored Field-Programmable Gate Arrays (FPGAs) for acceleration from multiple aspects [50], including complex query [84], distributed computation [34], and benchmarking tools [77]. TCUDB [31] transforms queries into matrix operations on Tensor Core Unit (TCU), which costs too much space and computation to support large-scale data, and is inherently different from our work based on compact 1-dimensional vectors.

Deconstructed/Hardware-independent Databases. Deconstructed database assembled from reusable and extensible components was recently proposed [35, 54] to avoid duplicate development efforts in extending systems to accommodate new hardware. Velox [53] is such a database that provides components used to unify existing computation engines at Meta. Apache Calcite [8] provides common components such as query languages, execution, and optimization for assembling new data systems and leaves data storage and management to specialized engines. Apache Arrow DataFusion [39] only requires some configuration lines to integrate with other systems, which has been integrated into many other products and projects [21, 32, 56, 63, 72]. However, these deconstructed systems/libraries target on CPU, which still need development efforts to make them compatible with emerging hardware accelerators. The only hardware-independent framework is TQP [29], which can be deployed and executed across multiple hardware backends.

8 Conclusion

In this paper, we analyzed the gap between SQL and tensors, and provided guidelines on how to fill this gap. Following these guidelines, we have developed a new tensor-based query engine, TQEx, by bridging the gap from multiple aspects, including the storage and computation of strings of variable lengths, as well as the optimization of tensor programs for expensive join and aggregate operators. To deal with large-scale data, we have also extended our system to support multi-GPU computation. TQEx demonstrates a significant performance improvement over the state-of-the-art CPU and GPU based systems on the TPC-H benchmark, and it also exhibits portability across different types of hardware accelerators.

Acknowledgments

The work was supported in part by grants of National Natural Science Foundation of China (No. 62272353 and No. 62276193), and Huawei Cloud Database Innovation Lab. Yuanyuan Zhu and Hao Zhang are corresponding authors.

References

- [1] 2024. BlazingSQL. <https://github.com/BlazingDB>
- [2] 2024. HeavyDB. <https://www.heavy.ai>
- [3] 2024. Kinetica. <https://www.kinetica.com>
- [4] 2024. Profiler. <https://github.com/pytorch/pytorch/blob/main/torch/autograd/profiler.py>.
- [5] 2025. *Broadcasting semantics*. <https://pytorch.org/docs/stable/notes/broadcasting.html> access date: 2025/4/10.
- [6] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek Gordon Murray, Benoit Steiner, Paul A. Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A System for Large-Scale Machine Learning. In *OSDI*. 265–283.
- [7] Cagri Balkesen, Gustavo Alonso, Jens Teubner, and M. Tamer Özsu. 2013. Multi-core, main-memory joins: sort vs. hash revisited. *PVLDB* 7 (2013), 85–96.
- [8] Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J. Mior, and Daniel Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. In *SIGMOD*. 221–230.
- [9] Spyros Blanas, Yinan Li, and Jignesh M. Patel. 2011. Design and evaluation of main memory hash join algorithms for multi-core CPUs. In *SIGMOD*. 37–48.
- [10] Nils Boeschen and Carsten Binnig. 2022. GaccO - A GPU-accelerated OLTP DBMS. In *SIGMOD*. 1003–1016.
- [11] Shekhar Borkar and Andrew A. Chien. 2011. The future of microprocessors. *Commun. ACM* 54, 5 (2011), 67–77.
- [12] Robert S. Boyer and J. Strother Moore. 1977. A fast string searching algorithm. *Commun. ACM* 20, 10 (oct 1977), 762–772.
- [13] Sebastian Breß and Gunter Saake. 2013. Why it is time for a HyPE: A hybrid query processing engine for efficient GPU coprocessing in DBMS. *PVLDB* 6, 12 (2013), 1398–1403.
- [14] Jiashen Cao, Rathijit Sen, Matteo Interlandi, Joy Arulraj, and Hyesoon Kim. 2023. GPU Database Systems Characterization and Optimization. *PVLDB* 17, 3 (2023), 441–454.
- [15] Jiaping Cao, Le Xu, Man Lung Yiu, Jianbin Qin, and Bo Tang. 2025. GPH: An Efficient and Effective Perfect Hashing Scheme for GPU Architectures. *SIGMOD* 3 (2025), 165:1–165:26.
- [16] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. 2015. MXNet: A Flexible and Efficient Machine Learning Library for Heterogeneous Distributed Systems. *CoRR* abs/1512.01274 (2015).
- [17] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *OSDI*. 578–594.
- [18] Jatin Chhugani, Anthony D. Nguyen, Victor W. Lee, William Macy, Mostafa Hagog, Yen-Kuang Chen, Akram Baransi, Sanjeev Kumar, and Pradeep Dubey. 2008. Efficient implementation of sorting on multi-core SIMD CPU architecture. *PVLDB* 1 (2008), 1313–1324.
- [19] Periklis Chrysogelos, Manos Karpapothakis, Raja Appuswamy, and Anastasia Ailamaki. 2019. HetExchange: Encapsulating heterogeneous CPU-GPU parallelism in JIT compiled engines. *PVLDB* 12, 5 (2019), 544–556.
- [20] John Cieslewicz, Kenneth A. Ross, Kyoho Satsumi, and Yang Ye. 2010. Automatic contention detection and amelioration for data-intensive operations. In *SIGMOD*. 483–494.
- [21] Coralogix. 2024. Coralogix - Full-Stack Observability Platform with In-Stream Data Analytics. <https://coralogix.com>
- [22] Transaction Processing Performance Council. 2005. Transaction processing performance council. *Web Site*, <http://www.tpc.org> (2005).
- [23] Maxime Crochemore, Thierry Lecroq, Artur Czumaj, Leszek Gasieniec, Stefan Jarominek, Wojciech Plandowski, and Wojciech Rytter. 1992. Speeding Up Two String-Matching Algorithms. In *STACS*. 589–600.
- [24] DBpedia. 2025. DBpedia. <https://www.dbpedia.org>. Accessed: 2025-07-15.
- [25] The Economist. 2010. Data, Data, Everywhere - A Special Report on Managing Information. *The Economist* (2010). <https://www.economist.com/special-report/2010/02/27/data-data-everywhere>
- [26] Ian J. Egielski, Jesse Huang, and Eddy Z. Zhang. 2014. Massive atomics for massive parallelism on GPUs. In *ISMM*. 93–103.
- [27] Henning Funke, Sebastian Breß, Stefan Noll, Volker Markl, and Jens Teubner. 2018. Pipelined Query Processing in Coprocessor Environments. In *SIGMOD*. 1603–1618.
- [28] Henning Funke and Jens Teubner. 2020. Data-parallel query processing on non-uniform data. *PVLDB* 13 (2020), 884–897.
- [29] Dong He, Supun Chathuranga Nakandala, Dalitso Banda, Rathijit Sen, Karla Saur, Kwanghyun Park, Carlo Curino, Jesús Camacho-Rodríguez, Konstantinos Karanasos, and Matteo Interlandi. 2022. Query Processing on Tensor Computation Runtimes. *PVLDB* 15, 11 (2022), 2811–2825.

- [30] Max Heimel, Michael Saecker, Holger Pirk, Stefan Manegold, and Volker Markl. 2013. Hardware-Oblivious Parallelism for In-Memory Column-Stores. *PVLDB* 6, 9 (2013), 709–720.
- [31] Yu-Ching Hu, Yuliang Li, and Hung-Wei Tseng. 2022. TCUDB: Accelerating Database with Tensor Processors. In *SIGMOD*. 1360–1374.
- [32] InfluxData Inc. 2024. InfluxDB – open source time series, metrics, and analytics database. <https://influxdata.com/>
- [33] Hiroshi Inoue, Takao Moriyama, Hideaki Komatsu, and Toshio Nakatani. 2007. AA-Sort: A New Parallel Sorting Algorithm for Multi-Core SIMD Processors. In *PACT*. 189–198.
- [34] Zsolt István, David Sidler, and Gustavo Alonso. 2017. Caribou: Intelligent distributed storage. *PVLDB* 10, 11 (2017), 1202–1213.
- [35] Amandeep Khurana and Julien Le Dem. 2018. The Modern Data Architecture: The Deconstructed Database. *login Usenix Mag.* 43, 4 (2018), 37–40.
- [36] Changkyu Kim, Tim Kaldewey, Victor W. Lee, Eric Sedlar, Anthony D. Nguyen, Nadathur Satish, Jatin Chhugani, Andrea Di Blas, and Pradeep Dubey. 2009. Sort vs. Hash revisited: fast join implementation on modern multi-core CPUs. *PVLDB* 2 (2009), 1378–1389.
- [37] Donald E. Knuth, James H. Morris, Jr., and Vaughan R. Pratt. 1977. Fast Pattern Matching in Strings. *SIAM J. Comput.* 6, 2 (jun 1977), 323–350.
- [38] Ruihang Lai, Junru Shao, Siyuan Feng, Steven Lyubomirsky, Bohan Hou, Wuwei Lin, Zihao Ye, Hongyi Jin, Yuchen Jin, Jiawei Liu, Lesheng Jin, Yaxing Cai, Ziheng Jiang, Yong Wu, Sunghyun Park, Prakash Srivastava, Jared Roesch, Todd C. Mowry, and Tianqi Chen. 2025. Relax: Composable Abstractions for End-to-End Dynamic Machine Learning. In *ASPLOS*. 998–1013.
- [39] Andrew Lamb, Yijie Shen, Daniël Heres, Jayjeet Chakraborty, Mehmet Ozan Kabak, Liang-Chi Hsieh, and Chao Sun. 2024. Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine. In *SIGMOD*. 5–17.
- [40] Viktor Leis, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2014. Morsel-driven parallelism: a NUMA-aware query evaluation framework for the many-core age. In *SIGMOD*. 743–754.
- [41] Jing Li, Hung-Wei Tseng, Chunbin Lin, Yannis Papakonstantinou, and Steven Swanson. 2016. Hippogriffdb: Balancing i/o and gpu bandwidth in big data analytics. *PVLDB* 9, 14 (2016), 1647–1658.
- [42] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. 2021. The Deep Learning Compiler: A Comprehensive Survey. *IEEE Transactions on Parallel and Distributed Systems* 32, 3 (2021), 708–727.
- [43] Clemens Lutz, Sebastian Breß, Steffen Zeuch, Tilmann Rabl, and Volker Markl. 2020. Pump Up the Volume: Processing Large Data on GPUs with Fast Interconnects. In *SIGMOD*. 1633–1649.
- [44] Prashanth Menon, Todd C. Mowry, and Andrew Pavlo. 2017. Relaxed operator fusion for in-memory databases: making compilation, vectorization, and prefetching work together at last. *PVLDB* 11 (2017), 1–13.
- [45] Microsoft. 2022. ONNX Runtime. <https://github.com/microsoft/onnxruntime>
- [46] Abhishek Modi, Kaushik Rajan, Srinivas Thimmaiah, Prakhari Jain, Swinky Mann, Ayushi Agarwal, Ajith Shetty, Shahid K I, Ashit Gosalia, and Partho Sarthi. 2021. New query optimization techniques in the Spark engine of Azure synapse. *PVLDB* 15 (2021), 936–948.
- [47] Amadou Ngom, Prashanth Menon, Matthew Butrovich, Lin Ma, Wan Shen Lim, Todd C. Mowry, and Andrew Pavlo. 2021. Filter Representation in Vectorized Query Execution. In *DAMON*. Article 6, 7 pages.
- [48] NVIDIA. 2025. cuDF. <https://docs.rapids.ai/api/cudf/stable/>. Accessed: 2025-07-15.
- [49] NVIDIA. 2025. Nsight Compute. <https://developer.nvidia.com/nsight-compute>. Accessed: 2025-07-15.
- [50] Philippos Papaphilippou and Wayne Luk. 2018. Accelerating database systems using FPGAs: A survey. In *FPL*. 125–1255.
- [51] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *NIPS*. 8024–8035.
- [52] Johns Paul, Jiong He, and Bingsheng He. 2016. GPL: A GPU-based pipelined query processing engine. In *SIGMOD*. 1935–1950.
- [53] Pedro Pedreira, Orri Erling, Maria Basmanova, Kevin Wilfong, Laith Sakka, Krishna Pai, Wei He, and Biswapesh Chattopadhyay. 2022. Velox: Meta’s Unified Execution Engine. *PVLDB* 15, 12 (2022), 3372–3384.
- [54] Pedro Pedreira, Orri Erling, Konstantinos Karanasos, Scott Schneider, Wes McKinney, Satya R Valluri, Mohamed Zait, and Jacques Nadeau. 2023. The Composable Data Management System Manifesto. *PVLDB* 16, 10 (2023), 2679–2685.
- [55] Orestis Polychroniou and Kenneth A. Ross. 2014. A comprehensive study of main-memory partitioning and its application to large-scale comparison- and radix-sort. In *SIGMOD*. 755–766.
- [56] The Dask Project. 2024. The dask-sql Project. <https://dask-sql.readthedocs.io/en/latest/>
- [57] Pytorch. 2024. Accelerating PyTorch with CUDA Graphs. <https://pytorch.org/blog/accelerating-pytorch-with-cuda-graphs/>

- [58] PyTorch. 2025. Introduction to torch.compile. https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html. Accessed: 2025-04-15.
- [59] Mark Raasveldt and Hannes Mühleisen. 2020. Data Management for Data Science-Towards Embedded Analytics. In *CIDR*.
- [60] Jared Roesch, Steven Lyubomirsky, Marisa Kirisame, Logan Weber, Josh Pollock, Luis Vega, Ziheng Jiang, Tianqi Chen, Thierry Moreau, and Zachary Tatlock. 2019. Relay: A High-Level Compiler for Deep Learning. *CoRR* abs/1904.08368 (2019).
- [61] Viktor Rosenfeld, Sebastian Breß, and Volker Markl. 2023. Query Processing on Heterogeneous CPU/GPU Systems. *ACM Comput. Surv.* 55, 2 (2023), 11:1–11:38.
- [62] Stefan Schuh, Xiao Chen, and Jens Dittrich. 2016. An Experimental Comparison of Thirteen Relational Equi-Joins in Main Memory. In *SIGMOD*. 1961–1976.
- [63] Seafowl. 2024. Seafowl Postgres Accelerator. <https://seafowl.io/>
- [64] Anil Shanbhag, Samuel Madden, and Xiangyao Yu. 2020. A study of the fundamental performance characteristics of GPUs and CPUs for database analytics. In *SIGMOD*. 1617–1632.
- [65] Anil Shanbhag, Bobbi W. Yogatama, Xiangyao Yu, and Samuel Madden. 2022. Tile-based Lightweight Integer Compression in GPU. In *SIGMOD*. 1390–1403.
- [66] SIGOPS. 2020. The Increasing Heterogeneity of Cloud Hardware and What It Means for Systems. <https://www.sigops.org/2020/the-increasing-heterogeneity-of-cloud-hardware-and-what-it-means-for-systems/>
- [67] Evangelia A. Sitaridi and Kenneth A. Ross. 2016. GPU-accelerated string matching for database applications. *PVLDB* 25 (2016), 719–740.
- [68] P. D. Smith. 1991. Experiments with a very fast substring search algorithm. *Softw. Pract. Exper.* 21, 10 (sep 1991), 1065–1074.
- [69] Science Staff. 2011. Dealing with data. challenges and opportunities. *Science* 331, 6108 (2011), 692–693.
- [70] Statista. 2024. Volume of data/information created, captured, copied, and consumed worldwide from 2010 to 2025. <https://www.statista.com/statistics/871513/worldwide-data-created/>
- [71] Daniel M. Sunday. 1990. A very fast substring search algorithm. *Commun. ACM* 33, 8 (aug 1990), 132–142.
- [72] Synnada. 2024. Synnada Real-Time Data Platform. <https://www.synnada.ai/>
- [73] Jens Teubner, Gustavo Alonso, Cagri Balkesen, and M. Tamer Ozsu. 2013. Main-memory hash joins on multi-core CPUs: Tuning to the underlying hardware. In *ICDE*. 362–373.
- [74] Thomas N. Theis and H.-S. Philip Wong. 2017. The End of Moore’s Law: A New Beginning for Information Technology. *Comput. Sci. Eng.* 19, 2 (2017), 41–50.
- [75] Jin Wang and Sudhakar Yalamanchili. 2014. Characterization and analysis of dynamic parallelism in unstructured GPU applications. In *IISWC*. 51–60.
- [76] Kaibo Wang, Kai Zhang, Yuan Yuan, Siyuan Ma, Rubao Lee, Xiaoning Ding, and Xiaodong Zhang. 2014. Concurrent analytical query processing with GPUs. *PVLDB* 7, 11 (2014), 1011–1022.
- [77] Zeke Wang, Hongjing Huang, Jie Zhang, and Gustavo Alonso. 2020. Shuhai: Benchmarking high bandwidth memory on fpgas. In *FCCM*. 111–119.
- [78] Daniel S. Weld, Raphael Hoffmann, and Fei Wu. 2009. Using Wikipedia to bootstrap open information extraction. *SIGMOD Rec.* 37 (2009), 62–68.
- [79] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52 (2009), 65–76.
- [80] Bowen Wu, Dimitrios Koutsoukos, and Gustavo Alonso. 2025. Efficiently Processing Joins and Grouped Aggregations on GPUs. *SIGMOD* 3 (2025), 27 pages.
- [81] Yang Ye, Kenneth A. Ross, and Norases Vespapunt. 2011. Scalable aggregation on multicore processors. In *DaMoN*. 1–9.
- [82] Bobbi W. Yogatama, Weiwei Gong, and Xiangyao Yu. 2022. Orchestrating Data Placement and Query Execution in Heterogeneous CPU-GPU DBMS. *PVLDB* 15 (2022), 2491–2503.
- [83] Yuan Yuan, Rubao Lee, and Xiaodong Zhang. 2013. The Yin and Yang of processing data warehousing queries on GPU devices. *PVLDB* 6, 10 (2013), 817–828.
- [84] Jinyu Zhan, Wei Jiang, Ying Li, Juntong Wu, Jianping Zhu, and Jinghuan Yu. 2021. Accelerating queries of big data systems by storage-side CPU-FPGA co-design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 7 (2021), 2128–2141.
- [85] Jingren Zhou, Per-Ake Larson, and Ronnie Chaiken. 2010. Incorporating partitioning and parallel plans into the SCOPE optimizer. In *ICDE*. 1060–1071.

Received April 2025; revised July 2025; accepted August 2025